

UNIT:4- JAVA PACKAGES & EXCEPTION HANDLING

1. PART 1: PACKAGES IN JAVA

A **package in Java** is a namespace that organizes a group of related classes and interfaces into a hierarchical structure.

It helps in avoiding name conflicts and improves code reusability, maintainability, and security.

Key Features:

- Avoids class name conflicts
 - Provides access protection (public, private, protected)
 - Improves code organization
 - Supports reusability
-

2. Types of Packages

1. System (Built-in) Packages : System packages are predefined packages provided by Java API that contain ready-to-use classes and interfaces.

These packages help developers perform common tasks without writing code from scratch.

Examples:

- java.lang (default package)
- java.util
- java.io
- java.sql

Example:

```
import java.util.Scanner;
public class Test {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter number:");
        int num = sc.nextInt();
        System.out.println("Number is: " + num);
    }
}
```

2. User-Defined Packages: User-defined packages are created by programmers to group their own classes into a structured format.

They help in modular programming and better project organization.

3. Creating a Package

Syntax:

```
package package_name;
```

Example:

```
package mypack;
```

```
public class Hello {  
    public void show() {  
        System.out.println("Hello from package");  
    }  
}
```

Compile:

```
javac -d . Hello.java
```

4. Accessing a Package

Ways to Access:

1. Using import statement
 2. Using fully qualified name
-

1. Using import

Syntax:

```
import package_name.class_name;
```

Example:

```
import mypack.Hello;
```

```
public class Main {  
    public static void main(String[] args) {  
        Hello obj = new Hello();  
        obj.show();  
    }  
}
```

2. Using Fully Qualified Name

Example:

```
public class Main {  
    public static void main(String[] args) {  
        mypack.Hello obj = new mypack.Hello();  
        obj.show();  
    }  
}
```

5. Adding Classes to a Package

Multiple classes can be grouped under the same package by declaring the same package name in all class files.

Example:

```
package mypack;  
  
public class A {  
    public void display() {  
        System.out.println("Class A");  
    }  
}  
  
package mypack;  
  
public class B {  
    public void show() {  
        System.out.println("Class B");  
    }  
}
```

PART 2: EXCEPTION HANDLING IN JAVA

1. **Exception** : An **exception** is an unwanted or unexpected event that occurs during the execution of a program and disrupts the normal flow of instructions.

It usually happens at runtime due to errors like division by zero, invalid input, or accessing invalid memory.

TYPES OF EXCEPTIONS IN JAVA

1. **Checked Exceptions**: - Checked exceptions are those exceptions that are **checked at compile-time** by the compiler.

The programmer must handle these exceptions using try-catch or declare them using throws.

Key Points:

- Occur at **compile time**
- Must be handled or declared
- Compiler gives error if not handled

Examples:

- IOException
- SQLException
- FileNotFoundException

Basic Program:

```
import java.io.*;
```

```
public class CheckedDemo {  
    public static void main(String[] args) {  
  
        try {  
            FileReader file = new FileReader("test.txt");  
        }  
        catch (FileNotFoundException e) {
```

```
        System.out.println("File not found");
    }
}
}
```

Output:

File not found

2. Unchecked Exceptions (Runtime Exceptions):-

Unchecked exceptions are those exceptions that occur **at runtime** and are not checked by the compiler.

Handling them is optional, but recommended to avoid program crash.

Key Points:

- Occur at **runtime**
 - Not checked at compile time
 - Handling is optional
-

Examples:

- `ArithmeticException`
 - `NullPointerException`
 - `ArrayIndexOutOfBoundsException`
-

Basic Program:

```
public class UncheckedDemo {
    public static void main(String[] args) {

        int a = 10 / 0; // runtime error

        System.out.println("Program continues...");
    }
}
```

Output:

Exception in thread "main" java.lang.ArithmeticException: / by zero

3. User-Defined Exceptions:-

User-defined exceptions are custom exceptions created by the programmer by extending the Exception class.

They are used to handle application-specific error conditions.

Basic Program:

```
class MyException extends Exception {  
    public MyException(String msg) {  
        super(msg);  
    }  
}  
  
public class CustomDemo {  
    public static void main(String[] args) {  
  
        try {  
            throw new MyException("Custom Error");  
        }  
        catch (MyException e) {  
            System.out.println(e.getMessage());  
        }  
    }  
}
```

output:

Custom Error

2. Exception Handling : Exception handling is a mechanism in Java that allows a program to handle runtime errors gracefully without terminating abruptly. It ensures that the program continues execution and maintains normal flow even after an error occurs.

3. Key Benefits of Exception Handling

- ✓Prevents abnormal program termination
- ✓Maintains normal program flow
- ✓Improves program reliability
- ✓Separates error-handling code from main logic
- ✓Helps in debugging and maintaining code

4. Keywords of Exception Handling

1. try:- The `try` block contains code that may cause an exception during execution.

It must be followed by either a `catch` block or a `finally` block.

Syntax:

```
try {  
    // risky code  
}
```

2. catch: The `catch` block is used to handle exceptions thrown from the `try` block. It defines how the program should respond when a specific exception occurs.

Syntax:

```
catch(ExceptionType e) {  
    // handling code  
}
```

FULL CODE STRUCTURE OF TRY-CATCH

Purpose: Handle runtime error

```
public class TryCatchDemo {  
    public static void main(String[] args) {  
  
        try {  
            int a = 10 / 0; // risky code
```

```

        }
        catch (ArithmeticException e) {
            System.out.println("Cannot divide by zero");
        }

        System.out.println("Program continues...");
    }
}

```

Output

Cannot divide by zero
Program continues..

3. finally:-The `finally` block always executes whether an exception occurs or not. It is mainly used for cleanup operations like closing resources.

Syntax:

```

finally {
    // cleanup code
}

```

EXAMPLE

```

        public class FinallyDemo {
        public static void main(String[] args) {

            try {
                int a = 10 / 0; // risky code
            }
            catch (ArithmeticException e) {
                System.out.println("Exception handled");
            }
            finally {
                System.out.println("Finally block executed");
            }

            System.out.println("Program continues...");
        }
    }
}

```

Output

Exception handled
Finally block executed
Program continues...

4. throw:- The `throw` keyword is used to explicitly create and throw an exception. It is mainly used to define custom error conditions in a program.

Syntax:

```

throw new ExceptionType("message");

```


Purpose: Manually create and throw an exception

Program

```
public class ThrowDemo {  
    public static void main(String[] args) {  
  
        int age = 15;  
  
        if (age < 18) {  
            throw new ArithmeticException("Not eligible to vote");  
        }  
  
        System.out.println("Eligible");  
    }  
}
```

Output

Exception in thread "main" java.lang.ArithmeticException: Not eligible to vote

1. Nested try:

A **nested try** block means placing one try block inside another try block.

It is used when different parts of code need separate exception handling or when one risky operation is inside another.

Syntax:

```
try {  
    // outer try  
  
    try {  
        // inner try  
    }  
    catch(Exception e) {  
        // inner catch  
    }  
  
}  
catch(Exception e) {  
    // outer catch  
}
```

Code Example:

```
public class NestedTryDemo {  
    public static void main(String[] args) {  
  
        try {  
            System.out.println("Outer try");  
  
            try {  
                int a = 10 / 0; // inner risky code  
            }  
            catch (ArithmeticException e) {  
                System.out.println("Inner catch: Division by zero");  
            }  
  
        }  
        catch (Exception e) {  
            System.out.println("Outer catch");  
        }  
  
        System.out.println("Program continues...");  
    }  
}
```

Output:

```
Outer try  
Inner catch: Division by zero  
Program continues...
```

2. Nested catch

Important: Java **does NOT allow catch inside catch directly**, so what we call *nested catch* actually means:
handling an exception and then creating/handling another exception inside that catch block.

A **nested catch** refers to a situation where a catch block contains another try-catch structure.

It is used when an exception occurs while handling another exception.

Syntax:

```
try {  
    // risky code  
}  
catch(Exception e) {  
  
    try {  
        // code that may cause another exception  
    }  
    catch(Exception ex) {  
        // handle second exception  
    }  
  
}
```

Code Example:

```
public class NestedCatchDemo {  
    public static void main(String[] args) {  
  
        try {  
            int a = 10 / 0; // first exception  
        }  
        catch (ArithmeticException e) {  
            System.out.println("First exception handled");  
  
            try {  
                int arr[] = new int[3];  
                arr[5] = 10; // second exception  
            }  
            catch (ArrayIndexOutOfBoundsException ex) {  
                System.out.println("Second exception handled");  
            }  
        }  
  
        System.out.println("Program continues...");  
    }  
}
```

Output:

First exception handled
Second exception handled
Program continues...

5. throws:- The throws keyword is used in method declaration to specify exceptions that a method may throw.

It informs the caller that the method can generate certain exceptions.

Syntax:

```
void methodName() throws ExceptionType {  
    // code  
}
```

Purpose: Declare exception in method

Program

```
public class ThrowsDemo {  
  
    static void checkAge(int age) throws Exception {  
        if (age < 18) {  
            throw new Exception("Not eligible");  
        } else {  
            System.out.println("Eligible");  
        }  
    }  
  
    public static void main(String[] args) {  
        try {  
            checkAge(15);  
        }  
        catch (Exception e) {  
            System.out.println(e.getMessage());  
        }  
    }  
}
```

Output

Not eligible